

Le code change...

La doc suit ? Trois skills en relai.

Emmanuel Sandorfi  LightOn



Ma première machine.

IBM PC XT.



Emmanuel Sandorfi

Software Engineer @ LightOn





Perdus dans la codebase, les questions ne se posent même plus.

Une convention qui a dérivé...

Un service refait il y a six mois, dont le pourquoi
a disparu...

Trois optimisations SQL qui ne sont pas
devenues une règle...



Vers une documentation technique pertinente et mise à jour

- Le coût ?
- Les lecteurs ?
- Les auteurs ?
- Le format ?



Architecture Decision Record (ADR)

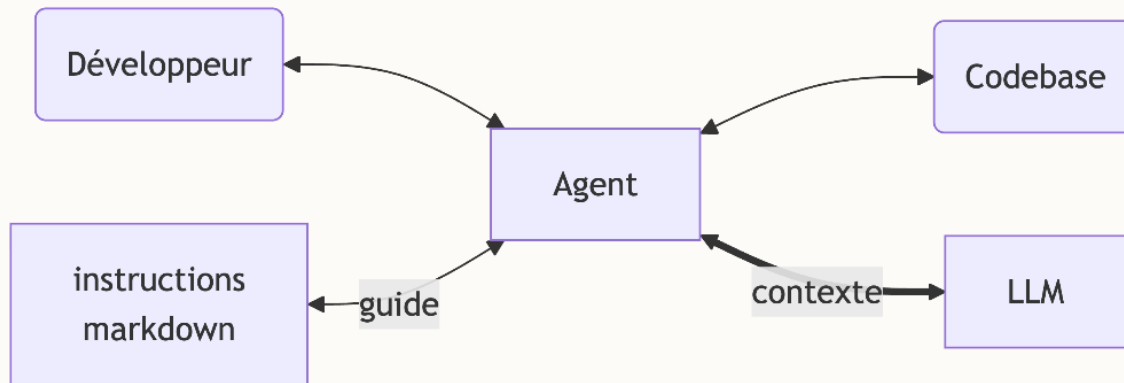
"Documenting architecture decision" - Michael Nygard
<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

Un ADR est une doc qui capture :

- **le contexte** — où on en était, ce qui poussait à décider
- **la décision** — ce qu'on a choisi
- **les conséquences** — ce que ça coûte, ce que ça gagne
- avec un **titre** et un **status**

L'agent de codage : Codex, Claude, OpenCode

Une boucle, un générateur, un dialogue.



L'enjeu tient en deux mots : **guide** et **contexte**. Sans l'un, l'agent improvise, sans l'autre, il invente.

Avec les deux, l'agent génère du contenu à haute valeur ajoutée.

Anatomie d'un skill

c'est un programme pour l'agent.

```
---
name: spec-adr
description: Generate an Architecture Decision Record for an app
---

# ADR Generator

Create an Architecture Decision Record for the **$ARGUMENTS** app.

## When To Use This

## Step 1: Validate

## Step 2: Gather

## Step 3: Identify and Propose

## Step 4: Generate

## Step 5: Place Rules

## Step 6: Verify Placement (MANDATORY)

## Step 7: Write

## Step 8: Confirm
```

Anatomie d'un skill

c'est un programme pour l'agent.

```
---
name: spec-adr
description: Generate an Architecture Decision Record for an app
---

# ADR Generator

Create an Architecture Decision Record for the **$ARGUMENTS** app.

## When To Use This

## Step 1: Validate

## Step 2: Gather

## Step 3: Identify and Propose

## Step 4: Generate

## Step 5: Place Rules

## Step 6: Verify Placement (MANDATORY)

## Step 7: Write

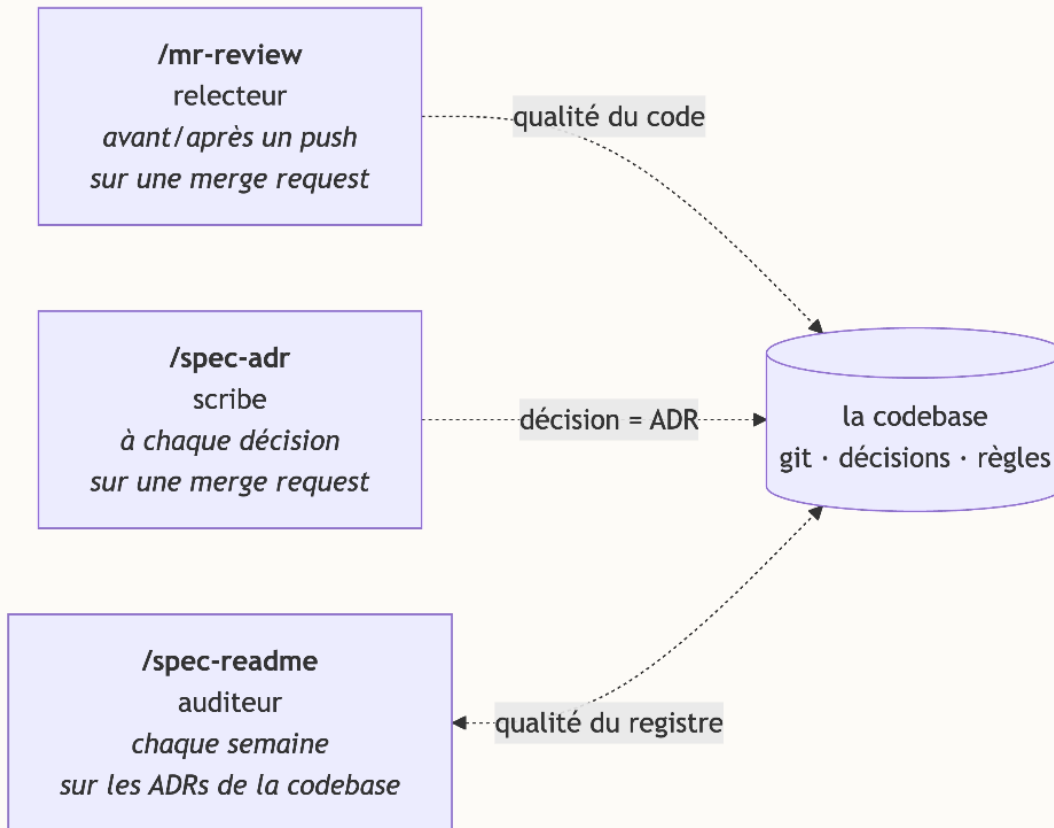
## Step 8: Confirm
```

Trois parties dans le fichier :

- **L'en-tête YAML** — pour le registre
- **Le mode d'emploi** — pour le dev
- **Les étapes 1 à 8** — pour le LLM

Un relai pour tenir un registre d'ADRs

Trois skills. Trois rôles. Trois moments.



claude /mr-review

A partir d'un ticket, d'une branche

```
• ---
MR Review: feature-search-debug-tooling - Keyword search debug tooling: search_context_template field + admin instrumentation #487

Branch: feature-search-debug-tooling -> main
Commits: 3, all backend - documents app
Ticket context: #487 - Operators had no way to inspect what text the keyword search engine actually indexed. Required re-running the
pipeline or querying the vector database directly. This MR persists a record-level keyword search text template and adds admin
instrumentation for instant inspection.

Overall assessment: Needs work before peer review - security issue with SENSITIVE_FIELDS must be addressed.

---
```

claude /mr-review

Ce qui va bien

```
---  
What looks good  
  
- Clean separation of template vs runtime text: build_search_template() is parallel to the existing build_search_text() with a {chunk}  
placeholder – stores record-level context once per record instead of N times per chunk.  
- Admin instrumentation is thorough: coloured keyword-index badge with relative age on chunks, yes/no filter for indexed status, collapsible  
fieldset on Record – all read-only and non-intrusive.
```


claude /mr-review

Les points bloquants et non-bloquants

```
---  
Things to address  
  
| [Must fix] documents/models/record.py:76 – search_context_template is not listed in SENSITIVE_FIELDS  
  
| Why it matters: The template contains the record's title, parsed filename, external path, and category labels – all of which are sensitive record metadata. Record.SENSITIVE_FIELDS currently lists ("filename", "title", "content", "file", "signature"). The redactor (redactors.py:47 via common.redactors.redact_resource) iterates SENSITIVE_FIELDS to replace values with REDACTED_PLACEHOLDER for unauthorized users. Since search_context_template is absent from the tuple, an admin user without record authorization will see title and filename redacted as "[Hidden]" but can still read them verbatim inside the template field – bypassing the entire redaction system.  
  
| The field is also shown in the Record admin change form (record.py:230) as a read-only collapsed fieldset, so it's directly visible in the UI.  
  
| Suggestion: Add "search_context_template" to Record.SENSITIVE_FIELDS:  
SENSITIVE_FIELDS = ("filename", "title", "content", "file", "signature", "search_context_template")
```

claude /mr-review

La synthèse

Checklist Summary

Category	Status	Notes
Security	Needs fix	<code>text_for_lexical_search</code> leaks document metadata to unauthorized admin viewers
Performance	Pass with suggestion	Consider deferring <code>text_for_lexical_search</code> on changelist (EMB0001)
Code Quality	Pass	Clean decomposition, good docstrings, DRY reuse of parsing functions
Testing	Pass	All new code paths covered; both positive and negative cases
ADR / Specs	Pass	EMB0006 permission rule followed; EMB0001 defer pattern could be extended
Correctness	Pass	Template construction correct; persistence gated on success; migration is safe (nullable AddField)

Would you like me to look deeper into any specific file or concern? Would you like me to help fix any of the issues found?

* Baked for 1m 47s

La décision émerge du code ?

Si oui, enrichir la codebase d'un ADR

claude /spec-adr propose : le dev arbitre

A partir d'une branche ou d'un commit

```
---
Proposed ADR: ADR-011 – Keyword Search Debug Visibility: Search Context Template and Admin Instrumentation

One-liner: Persist a record-level keyword search text template on the Record model so operators can instantly inspect what the search engine indexed, without re-running the pipeline or querying the vector database directly.

Key evidence from the diff:
- Support debugging required engineers to manually call build_search_text() in a shell – no self-service path for operators
- ContentChunk.category_search_indexed_at existed but was invisible in admin – no way to filter indexed vs unindexed chunks
- Record-level context (title, filename, path, category labels) is identical across all chunks of the same record but was never stored in a human-readable form

Decisions captured:
1. Store a {chunk} placeholder template on Record rather than duplicating context N times per chunk
2. Persist template only on successful indexing (errors == 0) so it's a trustworthy indicator
3. New build_search_template() as the canonical template builder (parallel to existing build_search_text())
4. Admin instrumentation: coloured badge, yes/no filter, collapsible fieldset
5. Permission gap fix: VIEW permission added to roles that had delete/edit

Want me to proceed with generating this ADR? Or would you like to adjust the scope (e.g., split the permission fix into a separate ADR)?

* Churned for 35s
```


La qualité augmente avec nos décisions

Governance checklist:

Check	Result
ADR in <code>documents/_specs/adr/</code> → rules in <code>documents/CLAUDE.md</code> ?	Pass – app-level ADR, app-level rules file
No code blocks in the rule?	Pass – terse constraint only; code examples stay in the ADR
No duplicates?	Pass – zero hits across all rules files
Promotion gate (2+ apps)?	N/A – single app, stays in <code>documents/CLAUDE.md</code>

Proposed rule – append to `documents/CLAUDE.md` section "Models & Admin":

Models & Admin

- ``Record.delete()`` / ``RecordQuerySet.delete()`` soft-deletes...
- Always use ``Record.all_objects`` in deletion services...
- ``defer("content", "owner__password")`` on Record list querysets...
- Cache repeated expensive calls on the admin instance...
- ``show_full_result_count = False`` on admin classes with large querysets.
- + ``search_context_template`` is persisted via ``build_search_template()`` only when all chunks index successfully (`errors == 0`). Never assemble keyword search record-level context manually – always use ``build_search_template()`` from ``documents/utils/search_utils.py``.
- ``values_list(..., flat=True).first()`` for single-value permission checks...

Why this rule is needed:

- Without it, a future contributor adding a new indexing path would likely write `Record.objects.filter(...).update(search_context_template=f"{title} {source} ...")` directly – duplicating the assembly logic and potentially persisting on error.
- The "only on success" constraint is non-obvious – the ingestion path and reindex path have different error-handling shapes, so the rule prevents inconsistency between them.

Notre gouvernance est codée dans le skill

Step 5: Place Rules

Rules derived from the ADR go to exactly ONE file. Follow the governance model (``.claude/rules/governance.md``):

Rule format: AI-actionable constraints

``<app>/CLAUDE.md`` is consumed by AI agents. Every rule must be a **terse constraint** for code generation:

- **DO/DON'T**: "Use X. Never Y."
- **WHEN/THEN**: "When X, do Y."
- **WHERE**: "Put X in Y. Never in Z."

NOT rules (→ stays in the ADR itself):

- Prose explaining WHY something was chosen
- Descriptive context about what exists
- Process instructions ("run this command after shipping")
- ADR links or references (the AI doesn't click links)

Rule placement logic

...

Is the rule specific to THIS app only?

→ `<app>/CLAUDE.md`

Is the rule platform-wide (applies to ALL apps)?

→ `.claude/rules/<domain>.md`

...

CRITICAL: Rules live in ONE place only. The ADR references them – it does NOT contain them.

How to add rules to ``<app>/CLAUDE.md``

``<app>/CLAUDE.md`` is organized by **coding context** (what you're doing), NOT by ADR origin. Standard sections:

...

Imports ← cross-app dependencies, platform boundary

Au cœur du skill : le template MADR

```
---
name: spec-adr
description: Generate an Architecture Decision Record for an app
---

## Step 4: Generate

Write the ADR in lightweight MADR format:

```markdown
<PREFIX>NNNN: <Noun Phrase Title>

Date: YYYY-MM-DD
Status: Proposal
Branch: `<branch-name>`

Context

2-4 sentences. What problem or need prompted this decision? What constraints existed?

Evidence

Concrete proof: failure counts, tracebacks, tables. Keep concise.

Decision

2-4 sentences. What was decided and how does it work?

Metrics

| | Before | After |
|--|-----|-----|
| Metric 1 | value | value |

Consequences
```

# Un ADR selon notre gouvernance

```
ADR0011: Keyword Search Debug Visibility – Search Context Template and Admin Instrumentation
```

```
Date: 2026-03-26
```

```
Status: Proposal
```

```
Branch: `feature-search-debug-tooling`
```

## ## Context

When keyword search results were unexpected (wrong ranking, missing records), operators had no way to inspect what text the search engine actually indexed. The only path was to re-run the ingestion pipeline with debug logging or query the vector database directly – both slow and requiring engineer access. Record-level context (title, filename, path, category labels) is assembled at indexing time and mixed with chunk text, but was never persisted in a human-readable form.

## ### Evidence

- Support tickets required engineers to manually call ``build_search_text()`` in a shell to see what the vector database received.
- Keyword reindex operations had no way to verify success per-record without querying the vector database's objects API.
- ``ContentChunk.category_search_indexed_at`` existed but was invisible in admin – operators couldn't filter indexed vs unindexed chunks.

## ## Decision

Store a record-level keyword search text `**template**` on ``Record.search_context_template`` – a ``TextField`` containing title, parsed filename, external path, and category labels with ``{chunk}`` as a placeholder for the chunk body. This template is persisted only when all chunks index successfully (`errors == 0`), making it a trustworthy indicator of what's actually in the vector database.

Admin instrumentation added:

- `**Record admin**`: collapsible "Keyword Search" fieldset showing the template (read-only).
- `**ContentChunk admin**`: ``SearchIndexedFilter`` (yes/no), coloured badge showing ``category_search_indexed_at`` with relative age.

## ### Metrics

	Before	After
Time to inspect search text for a record	~5 min (shell + pipeline)	~5 sec (admin click)
Ability to filter unindexed chunks	No	Yes (admin filter)
Record-level search context storage	Not persisted	1 row per record





`/spec-readme` rend le recul possible.

Chaque semaine, l'équipe ouvre le registre. Pas pour valider. Pour en parler.



# Trois mois de pratique.

## Ce qu'on a fait. Ce qu'on doit améliorer.

3 squads, ~10 domaines, 46 décisions documentées de février à mai

### Health Summary

Metric	Count	Details
Total ADRs	46	Across 10 domains
With agentic enforcement	40	Rules codified in <code>CLAUDE.md</code> or <code>.claude/rules/</code>
With CI enforcement	2	import-linter contracts (PRD0003, GRF0002)
No enforcement	6	IDM0001, IDM0002, VEC0002, VEC0006-stepwise, VEC0007, VEC0008
Duplicate IDs	6	VEC0006 x4, VEC0009 x2
Status inconsistency	1	PRD0001 uses "Approved" instead of "Accepted"
Stale proposals (>60 days)	12	See "Proposals Needing Review" below

# Trois mois de pratique.

Ce qu'on a trouvé : (n) Decisions Records

## Category Breakdown

Category	Count	ADRs
Architecture	23	IDM0001, IDM0002, IDM0005, DEL0001, SHR0001, SHR0002, SHR0003, VEC0004, VEC0005, VEC0006-stepwise, VEC0006-slots, VEC0007, VEC0009-search, VEC0010, PRD0003, PRD0005, PRD0009, PRD0010, PRD0011, PRD0013, GRF0001, GRF0002, BIL0001, RPC0001, CFG0001
Performance	6	VEC0001, VEC0002, VEC0003, VEC0006-celery, VEC0008, PRD0006
API Design	5	PRD0002, PRD0004, PRD0007, PRD0012, PRD0015
Security	4	IDM0003, IDM0004, VEC0006-perms, PRD0008
Testing	3	QAL0001, QAL0002, PRD0001
Reliability	2	VEC0009-zombie, GRF0003
Governance	1	GWY0001

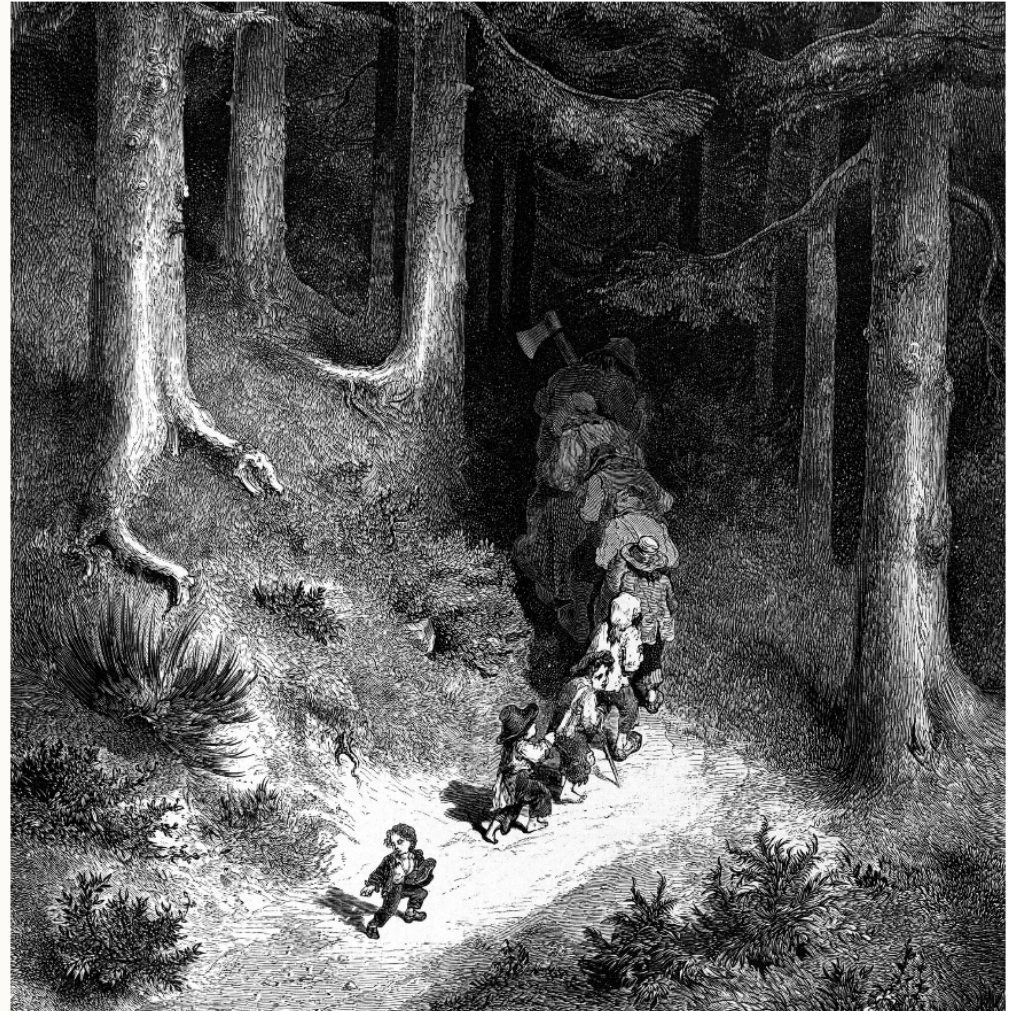
# Ce qu'on a appris

Le dev arbitre, pas l'agent.

Chaque décision devient un caillou blanc dans la codebase.

Pour que la doc suive, il faut la regarder.

Le skill reflète l'équipe qui l'a écrit.



# À votre tour.

Créer un skill, ça part d'un prompt.

Un premier pas : `github.com/esandorfi/spec-adr-starter-kit`

Generate an Architecture Decision Record from a git diff, versioned alongside the merge request, using a coding agent (Claude Code, OpenCode, Codex, etc.).



**Emmanuel Sandorfi**

[linkedin.com/in/sandorfi](https://linkedin.com/in/sandorfi)

Software Engineer

[github.com/esandorfi](https://github.com/esandorfi)



LightOn

[lighton.ai](https://lighton.ai)

